Agile Development Retrospective

Kyle DeJong | Travel application case study | Original reflection December 14, 2025

This reflection examines a simulated travel application project completed as undergraduate coursework. I wrote from the assigned Scrum Master perspective, considering requirements, changing priorities, communication, and the suitability of Scrum. ChadaTech and its travel client belong to the supplied fictional scenario.

The main lesson was that short feedback cycles are useful only when the work is clear enough to evaluate. The scenario's shift toward wellness travel, questions about profile preferences, and discussion of acceptance criteria provide concrete examples. This repository contains the written retrospective, not an application implementation or measured team-performance results.

Scrum Master Perspective

The Scrum Master role served as, in my own words, a "soft spine" for the project. Rather than acting overtly from a vertical hierarchy, the premise was to flatten and facilitate while still giving form to events. Such as enforcing time boxes, or focusing the team on outcomes over status reporting for the sake of it. This was not done through ordering, but encouraging, offering examples, leading by doing without demanding. In a sense, this was to act as a manager of the Scrum events themselves rather than employees. Sprint planning required realistic backlog selection. A team-specific "definition of ready" can help clarify items, but is not a formal Scrum requirement. The Daily Scrum needed to be short, consistent, and focused on outcomes and rooting out impediments. To take impediments as an example, the idea was to find what they may be as early as possible, make them highly visible, and assign ownership. The exposure principle helped defuse what could otherwise easily calcify as fiefs/blame, which then induce acting contrary to Agile principles. Thus, the events needed to be tended, and the team grew into the right shapes of their own volition. As Scrum Master, my purpose was to maintain cadence, focus, and transparency so the team could function effectively on the defined timeline.

The Daily Scrum belongs to the Developers, who inspect progress and adapt their plan. The Scrum Master supports effective Scrum practice rather than directing that meeting as a daily status report.

Product Owner Perspective

In this reflection, I considered the Product Owner primarily through stakeholder communication and backlog decisions. This does not require distance from the team's daily work. The focus was on constant, clear communication with the stakeholders to ensure project alignment and keeping the backlog well-ordered so the development team could deliver rapid, real value. In a metaphorical sense, this role maintains the correct shape of the entire project effort from where it meets the outside world, whereas the Scrum Master maintains the shape within itself. As an example, it required collecting user stories and gleaning the important details into usable formatting, "As a user, I want X." This translated actual user needs, based on interacting with them constantly, into workable development goals, rather than assumptions of what "should" be needed. With the backlog, discipline updating and pruning prevented scope creep by keeping focus on essentials rather than "nice to have" luxuries. This was informed by the stakeholders' input. Thus the Product Owner was the access point for external information which then shaped the entire project such that other team roles could mirror in their defined roles.

Testing Responsibilities

The course discussed Tester and Developer as separate perspectives. In the Scrum Guide, testing is part of the Developers' work rather than a fourth formal accountability. The distinction below describes responsibilities within the exercise.

The Tester's role was to ensure quality and clarity by taking the high-level user stories into measurable acceptance criteria that guided development and validation. As in, user stories often lacked fine detail, but carried intent. The Tester was responsible for turning that intent into testable conditions. With the team's "definition of done," tests became shared indicators of completion, because the entire team agreed on what "done" meant. Thus they also were called upon to communicate where gaps and ambiguity existed, as clarity, once received from the Product Owner, improved both the testing and the user stories. This was also seen in the discernment shown to said stories, wherein non-functional requirements and edge cases are often omitted for lack of assumed technical foreknowledge, and still must be found. Agile requires vigilance for this, as in the case of the travel client user stories where obvious, but unspoken, technical requirements were found in the arrangement of the user profile preference settings. It was not in the user story to include every toggle, reset, and ability to save, yet it was assumed by intent that they should be there.

Development Responsibilities

The development perspective was to remain focused on Scrum-style implementation while remaining flexible to priority changes. This was a significant departure from waterfall in that change was expected, and treated as good. Their job was to translate stories and acceptance criteria into the actual, working software. Thus every other role helps to support what the Developer creates, and in turn receives what is created to ensure it fits the project's shape. Developers thus needed to ask precise, thorough questions, to retain the high level communication and avoid situations of rework. During the Daily Scrum, priorities were to be clarified, blockers rooted out, and dependencies coordinated. Because they worked in incremental slices with said practices, pivots were relatively easy to execute because little had been set in stone. For example, when the travel client wished to change focus from top destination vacation packages to wellness retreats, it ultimately fell to the developer to make the change. And because work was done in those small slices, the scenario illustrated how a smaller scope could make revision more manageable. It did not establish a measured implementation cost.

From User Needs to Acceptance Criteria

When it came to user stories, there was a well-defined pipeline. To start, the Product Owner would gather stakeholder input. This would then become the framework, in human terms, for the user stories. "It would be nice to have an option to pick which kind of vacation I want, since I really like cruises, and don't care about city tours," becomes, "As a user, I want the ability to filter the kinds of vacations I like and dislike, such as cruises and city tours." This then enters into the backlog, which was refined to clarify which work needs to be done in the near-term, and only some elements for the long term. Stories were sized and prioritized as a result. Then acceptance criteria came from the Tester deriving the success conditions, which informed development and validation. Developers then implemented the stories within the boundaries of the sprint goals, and were considered complete when the acceptance criteria and shared Definition of Done were met. Thus there was a clear web of roles and duties in the creation and execution of the user stories. This iterative progress allowed them to move from the initial abstract needs to complete features by the end of a sprint.

Responding to Changed Priorities

One of the central comparisons in the exercise was how a project responds to change. The scenario shifted the travel packages from a list to a slideshow and changed the focus from general destinations to wellness retreats. Incremental work and an ordered backlog offered a way to discuss and accommodate those revisions. That does not mean changes are always trivial, or that a sequential process must wait until the end to address them. The useful distinction here was the opportunity to receive feedback and revisit assumptions while the work was still taking shape. During a Sprint, scope can be clarified with the Product Owner without endangering the Sprint Goal. The example should not imply unrestricted changes to a Sprint's purpose.

Communication and Clarification

Communication was the most commonly referenced element in our pilot run. Every role had duties towards it, and all were focused on avoiding siloing, opacity, and vagueness within the team (including the stakeholders). As examples, the Testers sent clarifying emails to the Product Owner on clarifying acceptance criteria. The Product Owner in turn took those specific questions and gave actionable answers. The Developer emailed the Product Owner and Tester on aligning priorities and technical expectations, and in doing so, bounded their questions and created shared understanding.

The request for alignment with other roles and clarity on the specifics of their role's tasks is exactly the intended effect of good communication. Thus, unnecessary, wasteful chatter was minimized. With the Daily Scrum as a fixed feature, silent drift was also headed off, and again, probed for blockers and any other problems before they could surface. With the communication tools as envisioned, there was specificity, a shared context, and timeliness. The natural gap between individuals and roles was bridged with reasonable measures and practices.

Backlogs and Shared Visibility

Organizational tools and Scrum events served as the scaffolding that allowed communication and role clarity to persist. The backlog was the central artifact of organization, it contained all work items and made explicit the trade-offs. With the Product Owner keeping it ordered by priority, development remained aligned with the stakeholder's desires and hampered scope creep by requiring decisions on what work should and should not be done. This transparency was key in maintaining trust among the disparate elements of the team and clients.

Task boards and information radiators further supported this paradigm by providing a visual representation of the work in progress. Shared, this allowed an immediate understanding of what work was underway, what was not, and what was finished. During the Daily Scrum, this acted as an anchor to keep focused on concrete progress/problems rather than diverging into abstracts. Thus, the tools reinforced the point of the event, to show progress towards the sprint goal and adapt plans.

Jira, or other equivalent Agile management tools, though we did not make use of them in the pilot, would have served to bind these concepts into a more permanent form. Jira would support sprint planning by allowing user stories to be decomposed into tasks, track commitments, and distinguish between completed and deferred work. Sprint Reviews would clearly identify completed backlog items, and feedback could be translated into new or adjusted entries. In the Retrospective, the tools would allow for a shared record of outcomes, to show what had actually occurred rather than subjective impression passed on (this paper being a case in point). The tools were not to replace communication, but reinforce transparency, inspection, and adaptation by ensuring the shared reality of the entire team.

Strengths and Limitations

Evaluating the Scrum/Agile approach for the travel client revealed a number of strengths. First and foremost was the adaptability. Technology needs change often and sometimes dramatically, the AI revolution has occurred only relatively recently, and being locked into a design for years before delivery that cannot bend and twist is to be stagnant while more agile competitors trapeze about. Because work was time-boxed into sprints and driven by the backlog, the scenario illustrated how changes could be considered without revisiting every project decision. Incremental delivery was intended to support early and continuous feedback, avoiding missing stakeholder expectations, and learning more of what exactly they wanted even in the midst of work. There was also the clear separation of responsibility between Scrum roles. The Product Owner was focused on value and priorities, the Scrum Master on cadence and flow, the Developers in implementation, and the Testers on validation, while quality remained a shared responsibility.

However, the pilot also exposed the necessary limitations of Scrum/Agile. By nature of the user story pipeline, they lacked fine detail, such as edge-cases and non-functional requirements, which necessitated a back and forth between the team and the stakeholders to avoid falling into implementation gaps. This approach also relied heavily on an assumed quality and timeliness of communication. Delays in stakeholder responses, unclear acceptance criteria, and even a team culture resistant to the Scrum Master's approach, could all easily stall out the momentum. And Scrum itself demands discipline to avoid drift and delay. In empowering the roles, there also comes responsibility. Without active work to keep events and the backlog flowing and tidied, the flexible structure could break down into disorder as nobody quite knows what exactly they should be doing, only that they should be doing something.

Despite these limitations, Scrum appeared well suited to the changing requirements in the ChadaTech travel scenario. The case supported using short feedback cycles, explicit priorities, and shared completion criteria to manage uncertainty. It did not prove that Scrum is universally better than waterfall, or establish measured delivery improvements. My recommendation within the simulation was to continue evaluating Scrum for similar projects, while paying close attention to communication, clarity, and the discipline needed to keep the backlog and events useful.

Reference

Ken Schwaber and Jeff Sutherland, The Scrum Guide, November 2020. https://scrumguides.org/scrum-guide.html